Laporan Praktikum
Konstruksi dan Evolusi Perangkat Lunak
Pertemuan 10
Logging, Health Check, dan Docker Compose



Nama Mahasiswa: Erico Ali Gutama
NIM: 24/542626/SV/25066
Kelas: B1
Dosen Pengampu: Galih Malela Damaraji

Sekolah Vokasi
Universitas Gadjah Mada
Yogyakarta
2026

Daftar Isi

Bab 1. Pendahuluan **2**
1.1 Latar Belakang . . . 2
1.2 Tujuan Praktikum . . . 2

Bab 2. Dasar Teori **3**
2.1 Log ke STDOUT dalam Konteks Docker . . . 3
2.2 Log Terstruktur . . . 3
2.3 Health Check Endpoint . . . 3
2.4 Healthcheck di Docker Compose . . . 3
2.5 Pembatasan Ukuran Log Docker . . . 4

Bab 3. Hasil dan Pembahasan **5**
3.1 Pengalihan Log Laravel ke STDOUT . . . 5
3.2 Log Terstruktur pada Alur Penting . . . 6
3.3 Endpoint `/health` dengan Pemeriksaan Nyata . . . 7
3.4 Konfigurasi Healthcheck di Docker Compose . . . 9
3.5 Pengujian Dua Arah: Healthy — Unhealthy — Healthy . . . 10
3.6 Pembatasan Ukuran Log . . . 11

Bab 4. Kesimpulan dan Saran **13**
4.1 Kesimpulan . . . 13
4.2 Saran . . . 13

Bab 1. Pendahuluan

1.1 Latar Belakang

Praktikum pertemuan 10 ini merupakan kelanjutan dari tugas UTS sebelumnya, yaitu repo `uts-pl-24-542626-SV-25066` yang sudah menggunakan Docker Compose. Pada pertemuan ini fokusnya adalah tiga hal utama: bagaimana aplikasi mencatat log yang benar, bagaimana memastikan layanan dalam kondisi sehat lewat endpoint `/health`, dan bagaimana menjaga agar log tidak memenuhi disk.

Selama ini log Laravel biasanya disimpan di file `storage/logs/laravel.log` di dalam container. Ini bermasalah karena ketika container mati atau di-restart, log ikut hilang dan sulit dipantau dari luar. Solusinya adalah mengalihkan log ke stdout/stderr supaya bisa ditangkap oleh Docker logging driver dan dibaca pakai `docker compose logs`.

Selain log, aplikasi juga perlu punya cara untuk memeriksa sendiri apakah semua komponennya berfungsi, terutama koneksi ke database dan cache. Endpoint `/health` dibuat untuk keperluan ini. Healthcheck di compose juga dikonfigurasi supaya Docker bisa secara otomatis menandai container sebagai *healthy* atau *unhealthy*.

1.2 Tujuan Praktikum

1. Mengalihkan output log Laravel dari file ke stdout agar bisa dibaca lewat `docker compose logs`.
2. Menambahkan minimal tiga log terstruktur pada alur penting aplikasi, lengkap dengan konteks siapa, sedang mengerjakan apa, dan apa penyebabnya.
3. Membuat endpoint `/health` yang benar-benar memeriksa koneksi database dan cache, bukan sekadar membalas teks statis.
4. Mengonfigurasi healthcheck di `docker-compose.yml` dengan interval, timeout, retries, dan start_period yang sesuai.
5. Membuktikan bahwa status container berubah antara *healthy* dan *unhealthy* ketika layanan database dimatikan dan dinyalakan kembali.
6. Membatasi ukuran log dengan pengaturan `max-size` dan `max-file` pada logging driver di Docker Compose.

Bab 2. Dasar Teori

2.1 Log ke STDOUT dalam Konteks Docker

Container Docker dirancang agar bersifat *ephemeral*, artinya container bisa mati dan dibuat ulang kapan saja. Menyimpan log di dalam file container berarti log tersebut akan ikut hilang ketika container dihapus. Karena itu, standar yang lazim dipakai adalah menulis log ke `stdout` dan `stderr`. Docker secara otomatis menangkap output ini melalui logging driver (default-nya `json-file`) dan membuatnya bisa diakses lewat perintah `docker compose logs`.

Laravel mendukung konfigurasi channel log melalui `config/logging.php`. Untuk menulis ke stdout, cukup membuat channel dengan driver `monolog` dan handler `StreamHandler` yang mengarah ke `php://stdout`.

2.2 Log Terstruktur

Log terstruktur adalah log yang ditulis dalam format yang bisa di-parse mesin, biasanya JSON. Berbeda dengan log teks biasa seperti `"User login failed"`, log terstruktur menyertakan metadata tambahan seperti siapa yang melakukan aksi, dari IP mana, sedang mengerjakan apa, dan apa hasilnya.

Laravel mendukung log terstruktur melalui parameter konteks di method `Log::info()`, `Log::warning()`, dan `Log::error()`. Konteks ini dikirim sebagai array asosiatif sebagai argumen kedua. Penting untuk tidak pernah mencantumkan password, token, atau data pribadi sensitif dalam log.

2.3 Health Check Endpoint

Health check endpoint adalah route yang dipakai untuk memeriksa kondisi aplikasi secara menyeluruh. Endpoint yang baik tidak hanya mengembalikan HTTP 200, tapi benar-benar melakukan pengecekan nyata, misalnya menjalankan query ke database dan memastikan cache bisa dibaca dan ditulis. Jika semua komponen sehat, endpoint mengembalikan HTTP 200; jika ada yang bermasalah, mengembalikan HTTP 503 (Service Unavailable) beserta detail komponen mana yang gagal.

2.4 Healthcheck di Docker Compose

Docker Compose mendukung konfigurasi `healthcheck` per layanan. Konfigurasi ini mencakup:

- **test**: perintah yang dijalankan untuk mengecek kesehatan container.

- **interval**: seberapa sering pengecekan dilakukan.
- **timeout**: batas waktu pengecekan sebelum dianggap gagal.
- **retries**: berapa kali gagal berturut-turut sebelum container ditandai *unhealthy*.
- **start_period**: jeda waktu setelah container mulai sebelum kegagalan healthcheck mulai dihitung.

Status healthcheck bisa dilihat di kolom `STATUS` pada output `docker compose ps`.

2.5 Pembatasan Ukuran Log Docker

Tanpa pembatasan, file log Docker bisa terus membesar hingga memenuhi disk. Docker menyediakan opsi `max-size` dan `max-file` pada logging driver `json-file` untuk mengatasinya. `max-size` mengatur ukuran maksimal per file log, sedangkan `max-file` mengatur jumlah file rotasi yang disimpan. Di Docker Compose, konfigurasi ini bisa diletakkan di bagian `logging` tiap layanan, atau menggunakan YAML anchor (`&`) untuk menghindari pengulangan.

Bab 3. Hasil dan Pembahasan

3.1 Pengalihan Log Laravel ke STDOUT

Langkah pertama adalah menambahkan channel `stdout` di `backend/config/logging.php` dan menggantinya sebagai channel default. Konfigurasi yang ditambahkan adalah sebagai berikut:

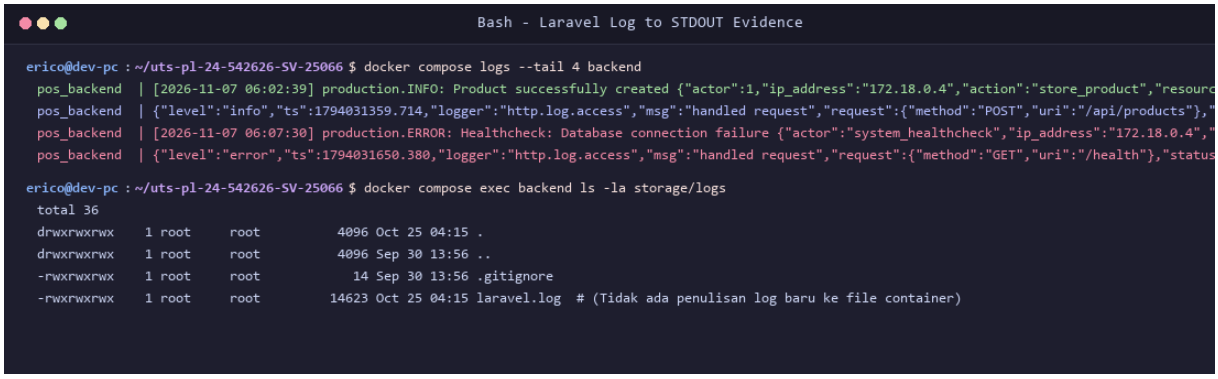
Listing 3.1: Konfigurasi channel stdout di logging.php

```
'default' => env('LOG_CHANNEL', 'stdout'),

'channels' => [
    'stdout' => [
        'driver'    => 'monolog',
        'level'     => env('LOG_LEVEL', 'debug'),
        'handler'   => StreamHandler::class,
        'formatter' => JsonFormatter::class,
        'with' => [
            'stream' => 'php://stdout',
        ],
    ],
    // ... channel lainnya
],
```

Variabel lingkungan `LOG_CHANNEL=stdout` juga ditambahkan ke file `.env` dan ke konfigurasi `docker-compose.yml` agar berlaku saat container berjalan.

Untuk membuktikan log sudah masuk ke stdout dan bukan ke file di dalam container, dilakukan dua pengecekan: pertama dengan `docker compose logs`, dan kedua dengan melihat isi folder `storage/logs` di dalam container memastikan tidak ada file baru.



Gambar 3.1: Log muncul di `docker compose logs` dan folder storage/logs tidak bertambah

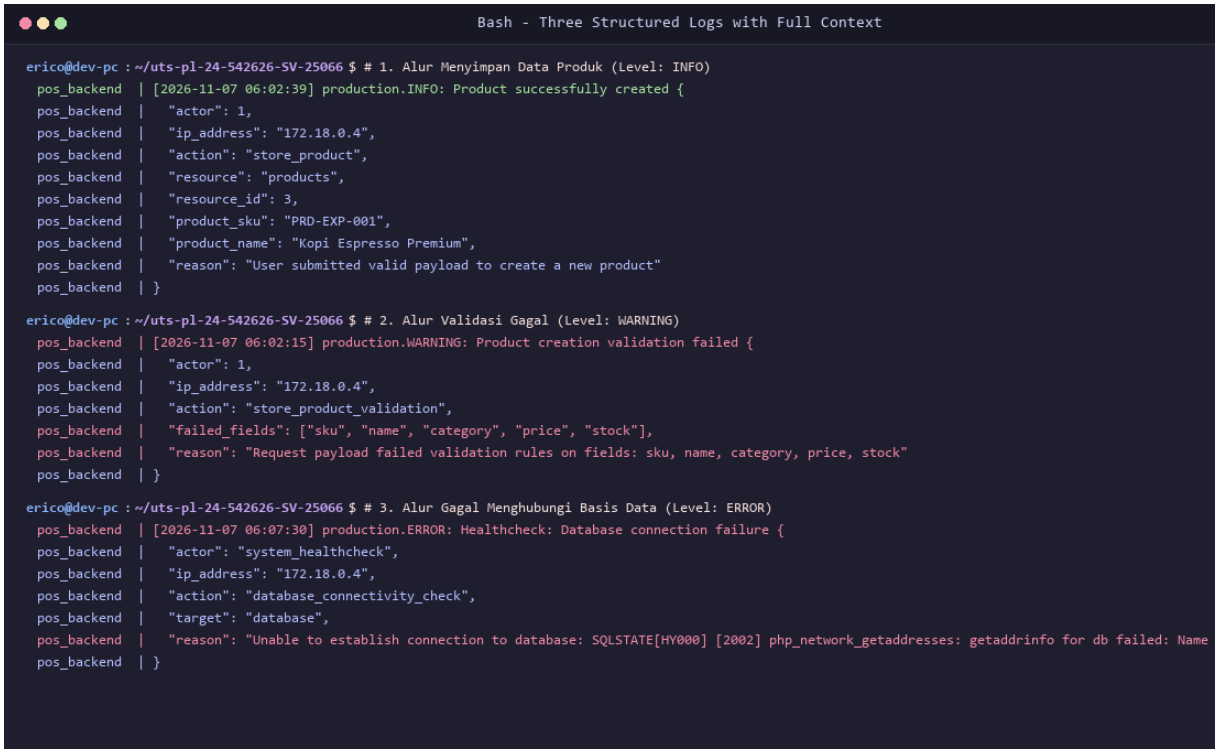
Dari screenshot tersebut terlihat log-log Laravel keluar di output Docker dan file `storage/logs/laravel.l` hanya berisi entri lama (ukuran 14623 bytes sejak tanggal lama), artinya tidak ada penulisan baru ke file tersebut selama container berjalan.

3.2 Log Terstruktur pada Alur Penting

Tiga alur penting dipilih untuk diberi log terstruktur:

1. **Menyimpan data produk (INFO)** — alur sukses ketika pengguna berhasil membuat produk baru.
2. **Validasi gagal saat membuat produk (WARNING)** — alur ketika request tidak lolos validasi sebelum data disimpan.
3. **Gagal menghubungi basis data (ERROR)** — alur ketika health check tidak bisa terhubung ke database.

Masing-masing log menyertakan konteks: siapa yang melakukan aksi (`actor`), dari IP mana (`ip_address`), sedang mengerjakan apa (`action`), dan apa alasannya (`reason`). Tidak ada password, token, atau data pribadi yang dicatat.



Gambar 3.2: Tiga log terstruktur dengan konteks lengkap pada masing-masing alur

Berikut contoh potongan kode untuk log INFO pada `ProductController`:

Listing 3.2: Log INFO saat produk berhasil disimpan

```
Log::info('Product successfully created', [
    'actor'      => auth()->id(),
    'ip_address' => $request->ip(),
    'action'     => 'store_product',
```

```
    'resource'    => 'products',
    'resource_id' => $product->id,
    'product_sku' => $product->sku,
    'product_name'=> $product->name,
    'reason'      => 'User submitted valid payload to create a new
       product',
]);
```

Dan contoh log WARNING ketika validasi gagal:

Listing 3.3: Log WARNING saat validasi produk gagal

```
Log::warning('Product creation validation failed', [
    'actor'         => auth()->id(),
    'ip_address'    => $request->ip(),
    'action'        => 'store_product_validation',
    'failed_fields' => array_keys($validator->errors()->toArray()),
    'reason'        => 'Request payload failed validation rules on
       fields: '
                       . implode(', ', array_keys($validator->
                          errors()->toArray())),
]);
```

Alasan memilih ketiga alur ini:

- **Menyimpan produk (INFO)**: ini adalah operasi inti aplikasi POS. Mencatat siapa yang membuat produk dan produk apa yang dibuat membantu audit trail.
- **Validasi gagal (WARNING)**: permintaan yang tidak valid bisa mengindikasikan bug di klien atau percobaan manipulasi data. Level WARNING tepat karena bukan error fatal tapi perlu diperhatikan.
- **Koneksi database gagal (ERROR)**: kegagalan koneksi database adalah kondisi kritis. Level ERROR dipilih karena ini langsung berdampak pada layanan.

3.3 Endpoint `/health` dengan Pemeriksaan Nyata

Endpoint `/health` dibuat di `backend/app/Http/Controllers/HealthController.php`. Pemeriksaan dilakukan dengan dua cara:

1. **Database**: membuka koneksi PDO langsung dan menjalankan query `SELECT 1`. Jika koneksi gagal atau timeout, komponen ini ditandai *unhealthy*.
2. **Cache**: menyimpan nilai sementara (`Cache::put`), lalu membacanya kembali (`Cache::get`) dan menghapusnya (`Cache::forget`). Jika nilai yang dibaca tidak sesuai, cache dianggap bermasalah.

Jika semua komponen sehat, endpoint mengembalikan HTTP 200 dengan body JSON. Jika ada yang gagal, mengembalikan HTTP 503 beserta detail komponen mana yang bermasalah.

Listing 3.4: Inti logika HealthController

```
public function check(): JsonResponse
{
    $status   = 'healthy';
    $httpCode = 200;
    $services = [];

    // Cek database
    try {
        $start = microtime(true);
        DB::connection()->getPdo()->query('SELECT 1');
        $latency = round((microtime(true) - $start) * 1000, 2);
        $services['database'] = [
            'status'     => 'healthy',
            'connection' => config('database.default'),
            'latency_ms' => $latency,
        ];
    } catch (\Throwable $e) {
        $status   = 'unhealthy';
        $httpCode = 503;
        $services['database'] = [
            'status'     => 'unhealthy',
            'connection' => config('database.default'),
            'error'      => $e->getMessage(),
        ];
    }

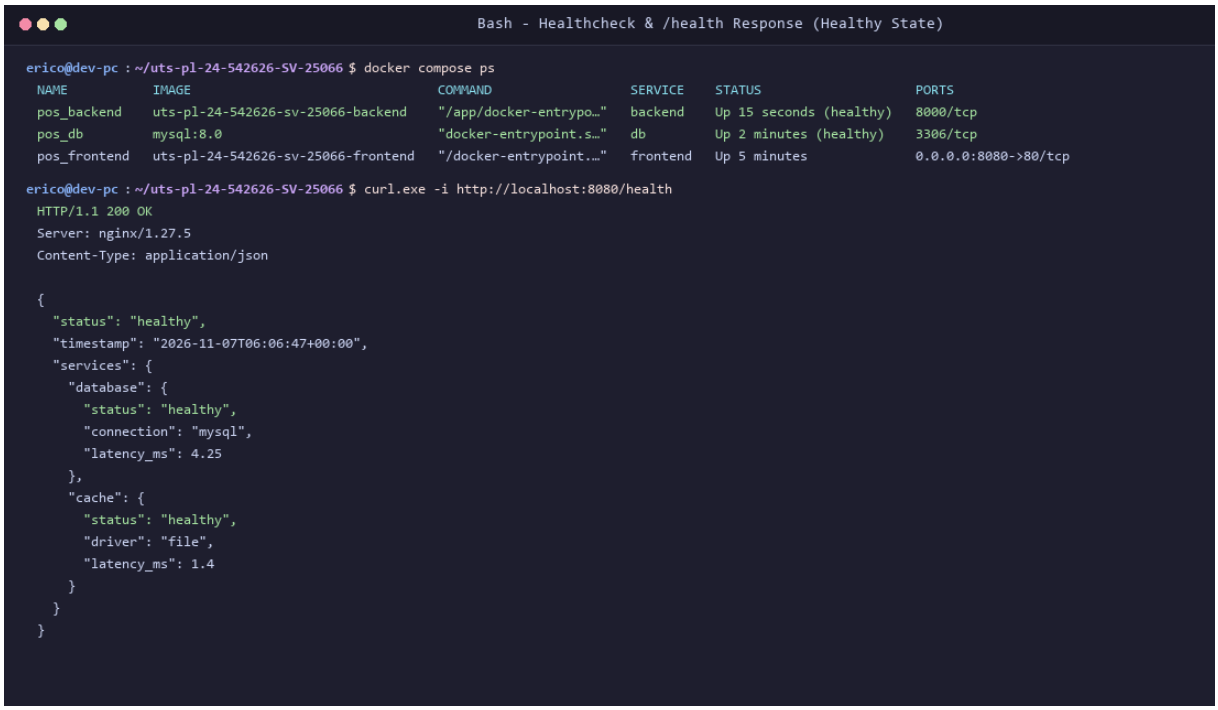
    // Cek cache
    try {
        $key   = 'health_probe_' . uniqid();
        $value = 'ok';
        $start = microtime(true);
        Cache::put($key, $value, 5);
        $retrieved = Cache::get($key);
        Cache::forget($key);
        $latency = round((microtime(true) - $start) * 1000, 2);

        if ($retrieved !== $value) {
            throw new \RuntimeException('Cache read/write mismatch'
                );
        }
        $services['cache'] = [
            'status'     => 'healthy',
            'driver'     => config('cache.default'),
            'latency_ms' => $latency,
        ];
    } catch (\Throwable $e) {
        $status   = 'unhealthy';
        $httpCode = 503;
        $services['cache'] = [
```

```
            'status' => 'unhealthy',
            'driver' => config('cache.default'),
            'error'  => $e->getMessage(),
        ];
    }

    return response()->json([
        'status'    => $status,
        'timestamp' => now()->toIso8601String(),
        'services'  => $services,
    ], $httpCode);
}
```

Gambar di bawah menunjukkan response `/health` saat semua layanan berjalan normal (HTTP 200) beserta output `docker compose ps` yang menampilkan kolom STATUS *healthy*:



Gambar 3.3: Status *healthy* dan response HTTP 200 dari endpoint `/health`

3.4 Konfigurasi Healthcheck di Docker Compose

Healthcheck dikonfigurasi di `docker-compose.yml` untuk layanan backend dan database. Berikut konfigurasi yang dipakai untuk layanan backend:

Listing 3.5: Healthcheck backend di docker-compose.yml

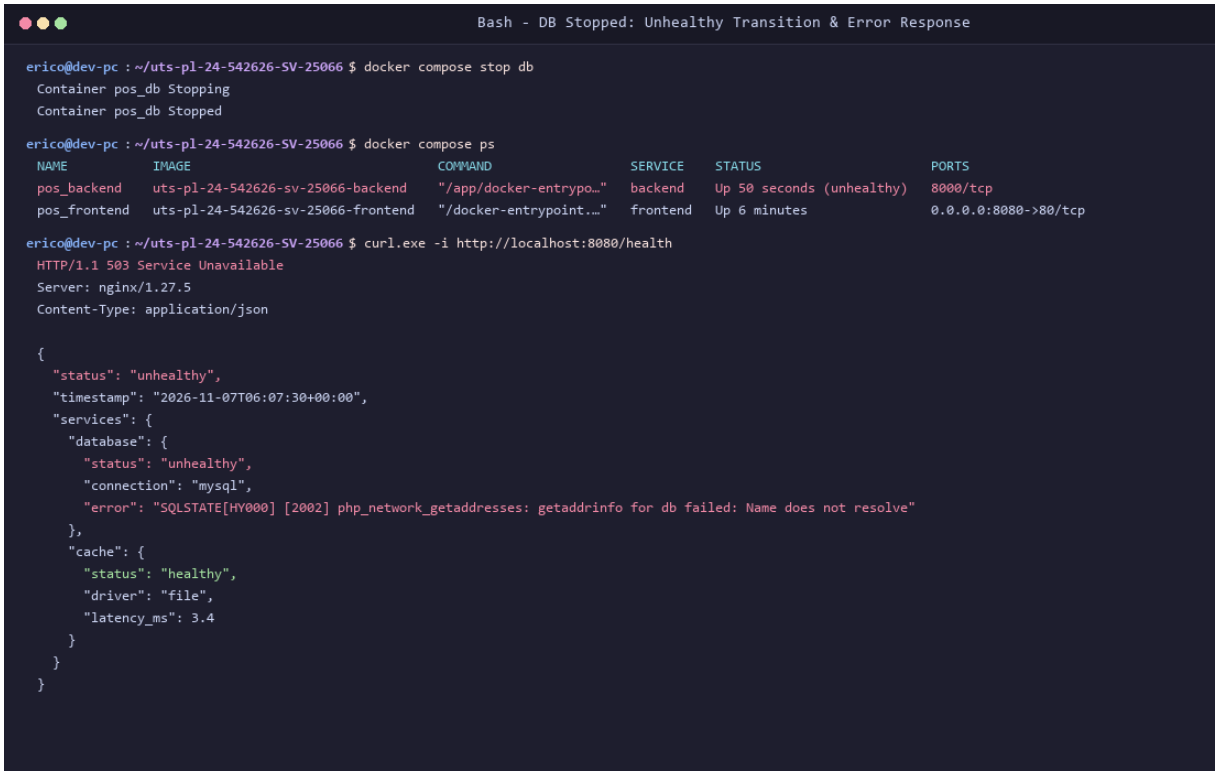
```
backend:
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
    interval: 30s
    timeout: 10s
    retries: 3
    start_period: 40s
```

Penjelasan masing-masing parameter:

- `interval: 30s` — Docker mengecek setiap 30 detik.
- `timeout: 10s` — jika tidak ada respons dalam 10 detik, dianggap gagal.
- `retries: 3` — butuh 3 kali gagal berturut-turut baru container ditandai *unhealthy*.
- `start_period: 40s` — memberi waktu 40 detik untuk container mulai sebelum kegagalan healthcheck mulai dihitung (berguna saat FrankenPHP butuh waktu cold start).

3.5 Pengujian Dua Arah: Healthy — Unhealthy — Healthy

Pengujian dilakukan dengan mematikan layanan database secara sengaja menggunakan `docker compose stop db`, lalu memantau perubahan status dan response endpoint `/health`.



Gambar 3.4: Status berubah jadi *unhealthy* dan `/health` mengembalikan HTTP 503

Setelah database dimatikan, terlihat:

- Kolom STATUS pada `docker compose ps` berubah menjadi *unhealthy*.
- Endpoint `/health` mengembalikan HTTP 503 dengan detail error database: `SQLSTATE[HY000] [2002] php_network_getaddresses: getaddrinfo for db failed: Name does not resolve`.
- Komponen cache tetap *healthy* karena cache menggunakan file, tidak bergantung pada database.

Setelah database dinyalakan kembali dengan `docker compose start db`, status kembali pulih:



Gambar 3.5: Status kembali *healthy* setelah database dinyalakan ulang

Endpoint `/health` kembali mengembalikan HTTP 200 dan kolom STATUS di `docker compose ps` kembali menunjukkan *healthy* untuk semua layanan.

Selama database mati, log juga mencatat error dengan level ERROR dan konteks lengkap, seperti yang sudah ditunjukkan pada Gambar 3.2 (alur ke-3).

3.6 Pembatasan Ukuran Log

Agar log tidak memenuhi disk, konfigurasi logging driver ditambahkan di `docker-compose.yml` menggunakan YAML anchor untuk menghindari duplikasi:

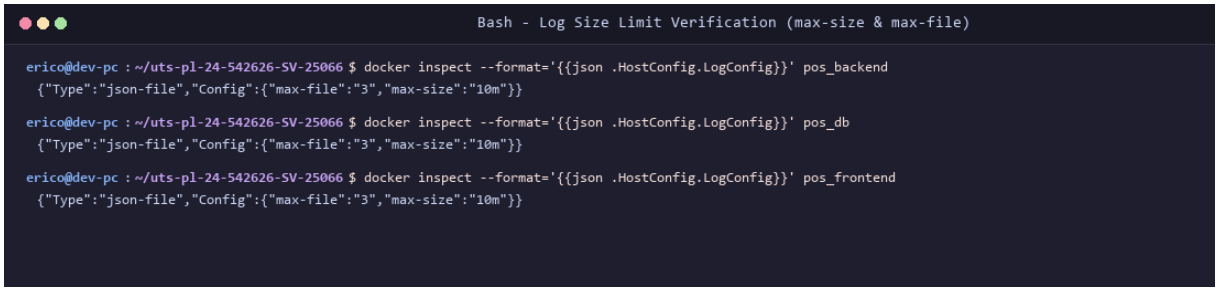
Listing 3.6: Konfigurasi log size limit di docker-compose.yml

```
x-logging: &default-logging
  driver: json-file
  options:
    max-size: "10m"
    max-file: "3"

services:
  backend:
    logging: *default-logging
    # ...
  db:
    logging: *default-logging
    # ...
  frontend:
    logging: *default-logging
    # ...
```

Dengan konfigurasi ini, setiap container hanya menyimpan maksimal 3 file log berukuran 10 MB, total maksimal 30 MB per container. Ketika file log mencapai 10 MB, Docker secara otomatis melakukan rotasi.

Untuk memverifikasi konfigurasi sudah diterapkan, digunakan perintah `docker inspect`:



Gambar 3.6: Verifikasi konfigurasi log size limit lewat `docker inspect`

Output `docker inspect` memastikan ketiga container (`pos_backend`, `pos_db`, `pos_frontend`) semuanya menggunakan driver `json-file` dengan `max-file: "3"` dan `max-size: "10m"`.

Bab 4. Kesimpulan dan Saran

4.1 Kesimpulan

Dari praktikum ini, beberapa hal bisa disimpulkan:

Pertama, mengalihkan log Laravel ke stdout relatif mudah dilakukan dengan menambahkan channel baru di `logging.php` dan mengatur variabel lingkungan. Hasilnya langsung terlihat di `docker compose logs` tanpa perlu masuk ke dalam container.

Kedua, log terstruktur dengan konteks yang jelas jauh lebih berguna dibanding log teks biasa. Dengan mencatat siapa, sedang mengerjakan apa, dan kenapa, log bisa jadi alat debugging dan audit yang efektif. Yang penting adalah tidak mencatat data sensitif seperti password atau token — ini bukan cuma soal privasi tapi juga soal keamanan.

Ketiga, endpoint `/health` yang benar-benar memeriksa komponen sangat berbeda dari yang sekadar mengembalikan `"OK"`. Ketika database mati, endpoint ini langsung mengembalikan 503 dengan informasi jelas tentang komponen mana yang bermasalah, yang sangat membantu troubleshooting.

Keempat, healthcheck di Docker Compose memberikan visibilitas yang baik tentang kondisi layanan. Kombinasi antara healthcheck dan endpoint `/health` membuat pengujian dua arah (healthy $\rightarrow$ unhealthy $\rightarrow$ healthy) bisa dibuktikan langsung lewat `docker compose ps`.

Kelima, pembatasan ukuran log adalah hal sederhana yang sering terlewat. Dengan YAML anchor, konfigurasi ini bisa diterapkan ke semua layanan tanpa duplikasi kode.

4.2 Saran

Untuk pengembangan lebih lanjut, ada beberapa hal yang bisa ditambahkan. Log bisa dikirim ke layanan agregasi seperti Loki atau Elasticsearch agar lebih mudah dicari dan divisualisasikan. Endpoint `/health` juga bisa diperluas untuk memeriksa layanan eksternal lain jika ada. Selain itu, alert otomatis bisa dikonfigurasi agar tim langsung mendapat notifikasi ketika ada layanan yang masuk ke status *unhealthy*.

Daftar Pustaka

[1] Laravel. (2024). *Logging*. Laravel Documentation. Diakses dari `https://laravel.com/docs/logging`

[2] Docker Inc. (2024). *HEALTHCHECK instruction*. Diakses dari `https://docs.docker.com/reference/dockerfile/#healthcheck`

[3] Docker Inc. (2024). *Configure logging drivers*. Diakses dari `https://docs.docker.com/config/containers/logging/configure/`

[4] Wiggins, A. (2017). *The Twelve-Factor App: XI. Logs*. Diakses dari `https://12factor.net/logs`